

Linear Algebra for Machine Learning

A Visual & Code-First Algebraic Primer

Linear algebra is the language of modern data representation and computation in machine learning. Features are represented as vectors, datasets as matrices, and operations as linear transformations. This guide provides a thorough, visually intuitive, and code-aligned overview of vector operations, matrix operations, transformations, eigenvalues/eigenvectors, and Singular Value Decomposition (SVD).

1. Vectors, Vector Spaces, and Projections

A vector is an ordered list of numbers representing a point or direction in space. The **dot product** ($\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos(\theta)$) measures the similarity between two vectors and is geometrically proportional to the cosine of the angle between them. Neurons compute activations using dot products: $a = \sum_i w_i x_i + b$.

The **projection** of vector $\mathbf{v}$ onto vector $\mathbf{u}$ splits $\mathbf{v}$ into a component parallel to $\mathbf{u}$ and an orthogonal component (see Figure 1). The formula is: $\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|^2} \mathbf{u}$. Vector projections are key to orthogonalization algorithms, linear regressions, and support vector machines.

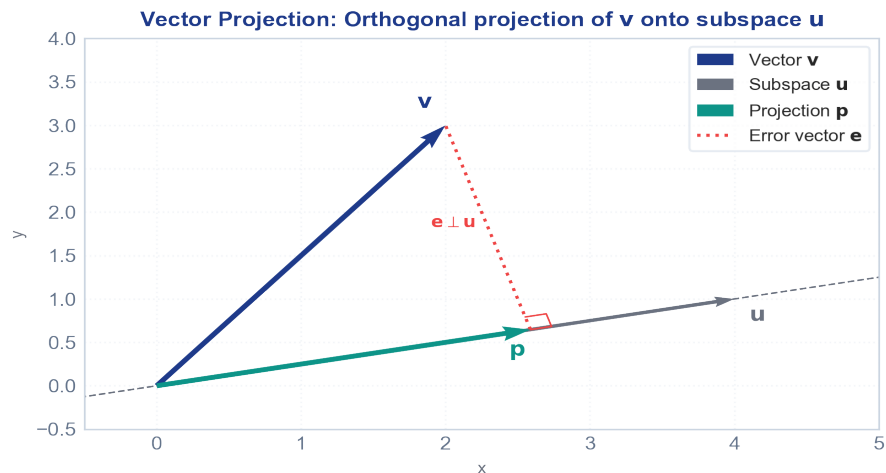


Figure 1: Geometric vector projection and orthogonal decomposition.

2. Matrices and Linear Transformations

A matrix is a 2D array of numbers that represents a linear transformation. Multiplying a vector by a matrix maps that vector to a new position in space. This transformation stretches, rotates, or shears the underlying coordinate grid (see Figure 2). Basis vectors $\mathbf{e}_1$ and $\mathbf{e}_2$ are mapped directly to the columns of the transformation matrix A . Neural network layers perform these transformations: $\mathbf{y} = XW + \mathbf{b}$.

The product $C = AB$ of two matrices is computed as: $C_{ij} = \sum_k A_{ik} B_{kj}$. Matrix transpositions ($(AB)^T = B^T A^T$) and matrix inversions ($A^{-1} A = I$) are fundamental mathematical operations in model weight derivations and dimensional scaling during backpropagation.

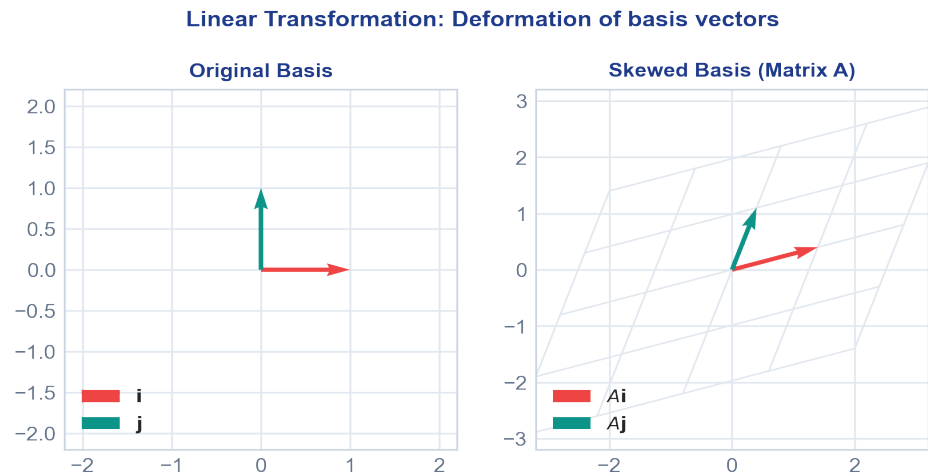


Figure 2: Matrix A distorting and mapping coordinate space.

3. Eigenvalues, Eigenvectors, and Singular Value Decomposition (SVD)

For a square matrix A , an **eigenvector** $\mathbf{v}$ is a special direction that does not change its heading when transformed by A ; it is only scaled by a scalar value, the **eigenvalue** λ : $A\mathbf{v} = \lambda\mathbf{v}$. In data analysis, eigenvectors of a covariance matrix point in the directions of maximum variance (Principal Components).

For non-square matrices, we generalize this concept using **Singular Value Decomposition (SVD)**: $X = U\Sigma V^T$, where U and V are orthogonal matrices and Σ contains the singular values ordered by magnitude (see Figure 3). SVD factorizes any matrix into three constituent operations, allowing for data compression and low-rank approximation.

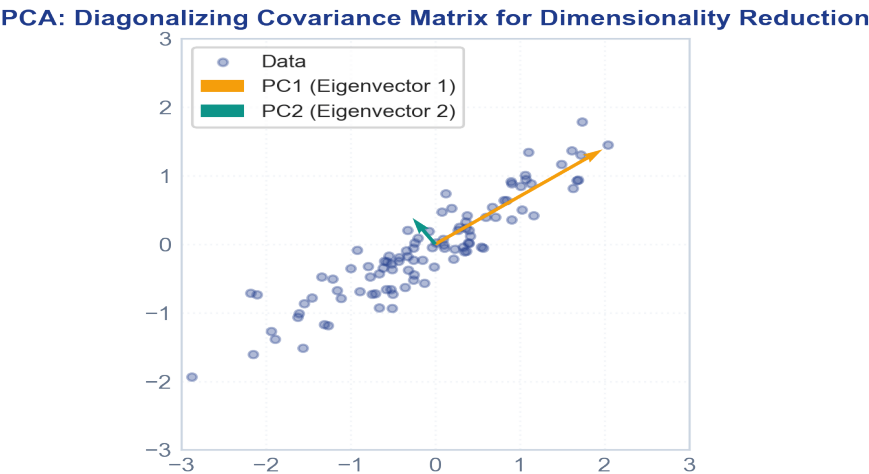


Figure 3: Eigenvectors of a centered covariance matrix (PC1 and PC2).

4. Machine Learning Integration: Singular Value Decomposition & PCA

SVD is the mathematical engine behind Principal Component Analysis (PCA). Given a centered data matrix X of size (m, n) , we find principal directions of variation by decomposing X . SVD decomposes the matrix directly without needing to calculate the covariance matrix, making it computationally stable. Below is the Python implementation of PCA from scratch using SVD:

```
import numpy as np

def pca_from_scratch(X, k=2):
    # 1. Center the data matrix
    X_centered = X - np.mean(X, axis=0)

    # 2. Apply Singular Value Decomposition (SVD)
    # U: left singular vectors, S: singular values, Vt: right singular vectors transpose
    U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)

    # Vt contains eigenvectors. Transpose Vt to get loading columns
    # The first k rows of Vt represent principal components
    W = Vt[:k, :].T # Transformation matrix (loading matrix)

    # 3. Project data into the reduced subspace
    X_projected = np.dot(X_centered, W)
    explained_variance = (S**2) / (len(X) - 1)
    return X_projected, W, explained_variance
```

ML INSIGHT: Matrix dimensions are the most common source of bugs in deep learning pipelines. Checking that weight matrix dimensions align properly during dot products (XW) is essential for successful implementations.

Linear Algebra Quick Reference Sheet

Essential vector-matrix properties and rules for machine learning.

Concept / Operation	Formula / Notation	ML Context / Application
Vector Dot Product	$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i = \ \mathbf{u}\ \ \mathbf{v}\ \cos(\theta)$	Computes projection, cosine similarity, and neuron activation input.
Vector Projection	$\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{u} \cdot \mathbf{v}}{\ \mathbf{u}\ ^2} \mathbf{u}$	Orthogonal decomposition of vectors; Gram-Schmidt orthogonalization.
Matrix Multiplication	$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$	Linear transformations in neural network layers ($\mathbf{XW} + \mathbf{b}$).
Eigenvalues & Eigenvectors	$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$	Identifies stationary directions under linear transformations.
Singular Value Decomposition	$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$	Matrix factorization used for PCA and low-rank approximation.
Covariance Matrix	$\mathbf{\Sigma}_{\text{COV}} = \frac{1}{m} \mathbf{X}^T \mathbf{X} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$	Captures feature variances and linear correlations.
Transpose Properties	$\begin{aligned} (\mathbf{AB})^T &= \mathbf{B}^T \mathbf{A}^T \\ (\mathbf{A} + \mathbf{B})^T &= \mathbf{A}^T + \mathbf{B}^T \end{aligned}$	Aligns dimensions of gradients during backpropagation.
Normal Equation	$\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$	Analytic closed-form solution to linear regression.